

CloudAge Global Services Pvt Ltd

Internal Technical Deployment Guide

Amazon Bedrock AgentCore Runtime — Financial Services AI Agent Platform. This guide summarizes the deployment strategy, architecture, and operational steps for launching an enterprise-grade intelligent finance assistant on AWS.

Prepared by

CloudAge Solutions
Architecture Team

Date

June 03, 2026

Version

1.2

Classification

Internal Use Only

Executive Summary

The CloudAge Financial Services AI Agent Platform is designed to accelerate a secure AWS deployment using Amazon Bedrock AgentCore, Strands Agents SDK, and managed AWS services. The architecture

enables a phased rollout from a single Budget Agent to a coordinated multi-agent ecosystem, delivering financial guidance, analytics, and automation with enterprise guardrails.

Project Scope: Launch a single-purpose Budget Assistant Agent, extend to a multi-agent financial analytics platform, deploy an AgentCore Gateway for secure API integration, and enable AWS observability, logging, and compliance controls.

- Deploy an intelligent budgeting assistant capable of natural-language financial assistance.
- Introduce a multi-agent orchestration layer for specialized budgeting, forecasting, and reporting.
- Provide secure AWS infrastructure and Bedrock guardrails for production readiness.
- Establish monitoring, logging, and operational runbooks for a production-grade rollout.
- Complete the initial deployment plan by June 30, 2026.

Project Overview

Business Context

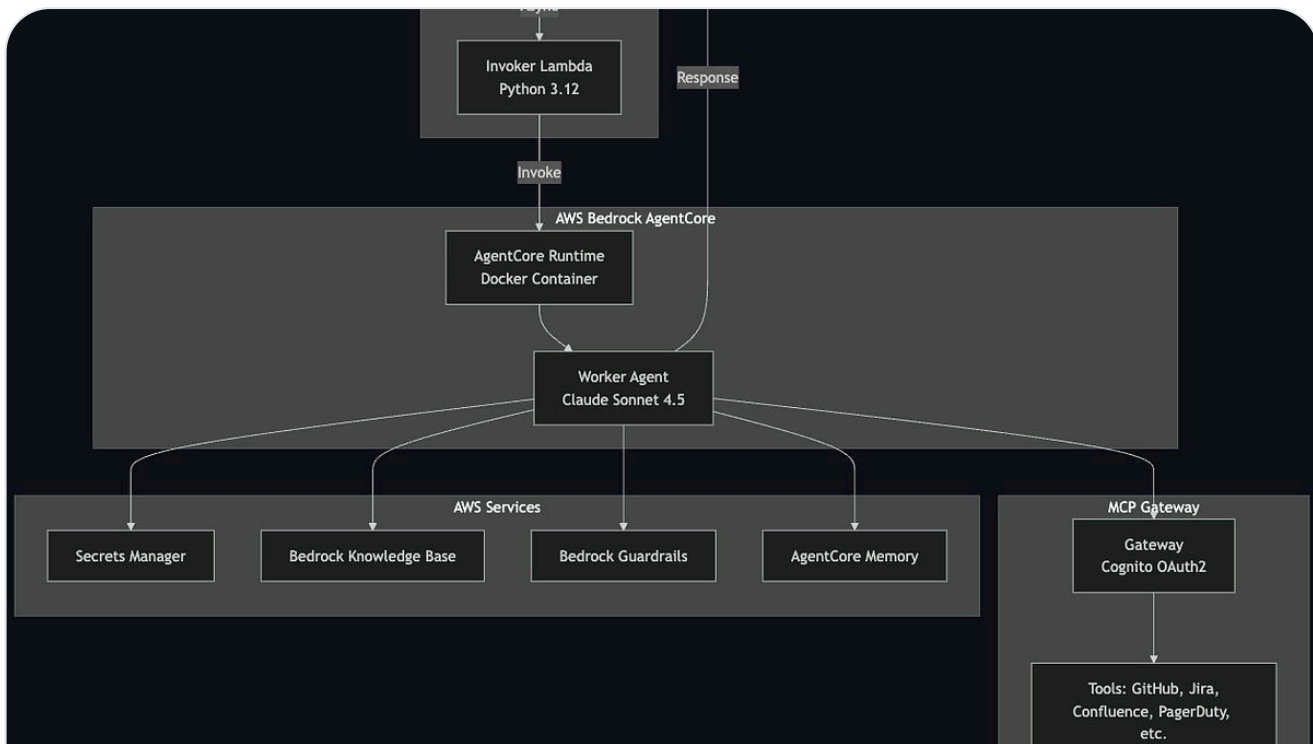
The target solution helps financial services customers manage budgets, evaluate spending patterns, and support investment decisions with a

conversational AI experience. It must interpret natural language, perform deterministic financial calculations, generate visualizations, and orchestrate multi-step workflows across agent roles.

- Understand complex, conversational finance queries with context-aware responses.
- Drive decisions using tool-based calculations, data lookups, and charting.
- Scale from a single Budget Agent to a broader financial analysis ecosystem.
- Use AWS managed services for security, reliability, and observability.

Solution Architecture

The architecture begins with a single-purpose Budget Agent and evolves into a multi-agent system with dedicated Financial Analysis and Orchestrator agents. A secure AgentCore Gateway provides centralized API access and routing, while AWS resources handle storage, logging, monitoring, and policy enforcement.



Architecture Overview: Progressive deployment from single agent to multi-agent orchestration on AWS.

Deployment Instructions

Prerequisites

Before deployment, ensure the environment is configured with the required AWS accounts, permissions, and local tooling.

AWS Account Setup

- Create a dedicated AWS account within the client organization.

- Provision IAM roles with least privilege access for Bedrock, S3, DynamoDB, Secrets Manager, CloudWatch, and Lambda.

Required Tools

- AWS CLI v2.19.0+
- AWS SAM CLI v1.100.0+
- Docker v24.0.0+
- Python 3.12+ with pip

Environment Configuration

```
export AWS_REGION=us-east-1  
export CLIENT_ENVIRONMENT=production  
export PROJECT_NAME=financial-agent-platform
```

Single Agent Deployment

Budget Agent Implementation

Core Components

Component	Configuration	Location
AgentCore Runtime	Docker containerized execution environment	<code>./agent-runtime/Dockerfile</code>
Foundation Model	Claude Sonnet 4.5 via Amazon Bedrock	<code>./config/bedrock-models.yml</code>
Custom Tools	Financial calculations, data visualization	<code>./tools/financial_tools.py</code>
Memory Layer	Conversation context and state persistence	<code>./memory/conversation_memory.py</code>
Security Layer	IAM policies, guardrails	<code>./security/agent_policies.json</code>

Deployment Steps

1. Set up the AgentCore Runtime

```
cd agent-runtime
docker build -t budget-agent-runtime .
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com
```

2. Configure Bedrock Foundation Model

```
aws bedrock create-model-configuration \
  --model-id "anthropic.claude-3-sonnet-20240229-v1:
  --region $AWS_REGION
```

3. Deploy Custom Tools

```
from tools.base import Tool

class BudgetCalculator(Tool):
    """Calculates 50/30/20 budget rule allocations"""

    def run(self, income: float, expenses: dict) -> dict:
        needs = income * 0.5
        wants = income * 0.3
        savings = income * 0.2
        return {
            "needs_allocation": needs,
            "wants_allocation": wants,
            "savings_allocation": savings,
            "expenses_by_category": expenses
        }
```

4. Configure Conversation Memory

```
aws dynamodb create-table \
  --table-name AgentConversationState \
  --attribute-definitions AttributeName=sessionId,AttributeType=String \
  --key-schema AttributeName=sessionId,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
  --region $AWS_REGION
```

5. Deploy Security Configuration

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeAgent"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:*"
    }
  ]
}
```

Multi-Agent System Deployment

Multi-Agent Architecture



Why Amazon Bedrock AgentCore Matters for AI Development

Learn how Amazon Bedrock AgentCore helps developers build, deploy, and scale intelligent AI agents securely using AWS-native tools and automation.

www.signiance.com



Multi-Agent Architecture with specialized agents and orchestration.

Specialized Agents Configuration

Agent Type	Purpose	Deployment Location
Budget Agent	Personal budgeting	<code>./agents/budget_agent/</code>

Agent Type	Purpose	Deployment Location
	and spending analysis	
Financial Analysis Agent	Investment research and portfolio management	<code>./agents/financial_analysis/</code>
Orchestrator Agent	Query routing and response synthesis	<code>./agents/orchestrator/</code>

Deployment Steps

1. Create Agent Definitions

```
budget_agent:
  name: "BudgetAgent"
  description: "Specializes in personal budgeting, s
  foundation_model: "anthropic.claude-3-sonnet-20240
  tools:
    - "budget_calculator"
    - "spending_analyzer"
    - "visualization_generator"

financial_analysis_agent:
  name: "FinancialAnalysisAgent"
  description: "Focuses on investment research, port
```

```

foundation_model: "anthropic.claude-3-sonnet-20240
tools:
  - "market_researcher"
  - "portfolio_analyzer"
  - "risk_assessor"

orchestrator_agent:
  name: "OrchestratorAgent"
  description: "Coordinates specialized agents and s
  foundation_model: "anthropic.claude-3-sonnet-20240
  tools:
    - "agent_router"
    - "response_synthesizer"
    - "validation_tool"

```

2. Implement Agent Communication Protocol

```

from typing import Dict, Any
from agents.config import AGENT_CONFIG
from tools.base import ToolResponse

class AgentCommunicator:
    """Handles communication between specialized age

    def __init__(self):
        self.agents = {}
        for agent_name, config in AGENT_CONFIG.items:
            self.agents[agent_name] = self._initiali

    def _initialize_agent(self, name: str, config: d

```

```
"""Initialize an agent instance"""
```

```
pass
```

```
def query_agent(self, agent_name: str, query: str)
```

```
    """Query a specific agent with context"""
```

```
    if agent_name not in self.agents:
```

```
        raise ValueError(f"Agent {agent_name} not found")
```

```
    return self.agents[agent_name].run(query, context)
```

3. Deploy Orchestrator Agent

```
sam build --template template.yaml
```

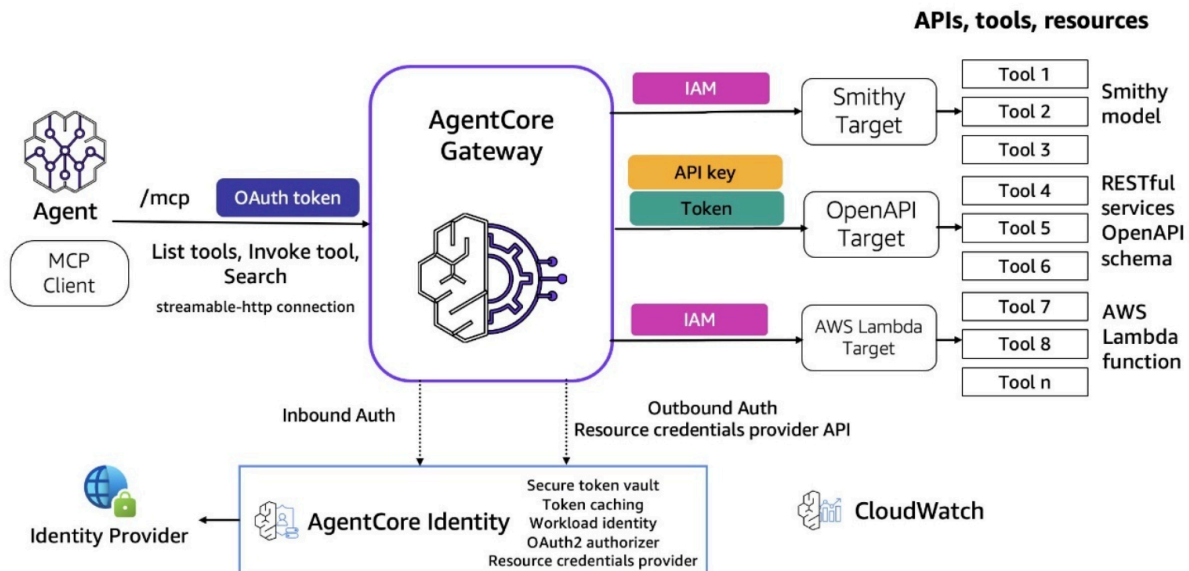
```
sam deploy --stack-name orchestrator-agent --region
```

4. Configure Multi-Agent Communication

Establish secure messaging channels between agents and the orchestrator using the AgentCore Gateway.

AgentCore Gateway Integration

Gateway Configuration



AgentCore Gateway architecture for MCC and API connectivity.

MCP Implementation

```
from typing import Dict, Any, Optional
from datetime import datetime

class MCPProtocol:
    """Standardized agent-to-tool communication protocol"""

    @staticmethod
    def create_request(
        agent_id: str,
        tool_name: str,
        parameters: Dict[str, Any],
        conversation_id: Optional[str] = None
    ) -> Dict[str, Any]:
        return {
```

```
"protocol_version": "1.0",
"agent_id": agent_id,
"tool_name": tool_name,
"parameters": parameters,
"conversation_id": conversation_id,
"timestamp": datetime.utcnow().isoformat()
}

@staticmethod
def parse_response(response: Dict[str, Any]) -> Dict[str, Any]:
    if response.get("status") != "success":
        raise Exception(f"Tool execution failed: {response.get('error')}")
    return {
        "result": response.get("result"),
        "metadata": response.get("metadata", {}),
        "execution_time": response.get("execution_time")
    }
```

API Gateway Integration

```
aws apigateway create-rest-api \
  --name "AgentCoreAPI" \
  --region $AWS_REGION

aws apigateway create-deployment \
  --rest-api-id [API_ID] \
```

```
--stage-name prod \  
--region $AWS_REGION
```

Observability & Monitoring

CloudWatch Configuration

Create dashboards to monitor agent performance, tool execution, health, and security events.

Dashboard	Key Metrics
Agent Performance	Response time, error rate, invocation count
Agent Health	Memory usage, CPU utilization, container health
Tool Performance	Tool execution success rate, latency
Security	Access attempts, policy violations

```
aws cloudwatch put-dashboard \  
--dashboard-name "FinancialAgent-Dashboard" \  
--dashboard-body file://dashboard-config.json \  
--region $AWS_REGION
```

X-Ray Tracing Configuration

```
{
  "Version": 1,
  "SamplingRule": {
    "RuleName": "AgentCoreRule",
    "ResourceARN": "*",
    "Priority": 1,
    "FixedRate": 0.1,
    "ReservoirSize": 100,
    "Host": "*",
    "HTTPMethod": "*",
    "URLPath": "*",
    "ServiceName": "AgentCore",
    "ServiceType": "*",
    "Attributes": {}
  }
}
```

Security & Compliance

Security Implementation Checklist

- ☒ Configure IAM roles with least privilege access
- ☒ Set up AWS Secrets Manager for sensitive credentials

- ✓ Enable AWS KMS for encryption at rest
- ✓ Implement Bedrock Guardrails for content safety
- ✓ Configure AWS WAF for API protection
- ✓ Set up AWS Shield for DDoS protection
- ✓ Enable CloudTrail for API activity logging
- ✓ Implement VPC isolation with private subnets
- ✓ Establish privacy and Responsible AI controls for model outputs and data access

Security Validation Steps

1. Run Security Scan

```
aws securityhub batch-enable-standards \
--standards-subscription-requests file://security-
```

2. Validate Guardrails Configuration

```
from botocore.exceptions import ClientError
import boto3

def validate_guardrails():
    """Validate Bedrock guardrails configuration"""
    try:
        bedrock_client = boto3.client('bedrock')
        guardrails = bedrock_client.list_guardrails(
            required_guardrails = ["content_moderation",
            for guardrail in required_guardrails:
                if guardrail not in [g['name'] for g in
```

```
        raise Exception(f"Required guardrail {guardrail} not found")
    print("Guardrails validation successful")
    return True
except ClientError as e:
    print(f"Guardrails validation failed: {str(e)}")
    return False
```

Deployment Checklist

Pre-Deployment Validation

- Verify AWS account permissions
- Confirm network configuration
- Validate Docker images are built and pushed
- Confirm required secrets are in Secrets Manager
- Confirm CloudWatch logging is configured
- Validate security configurations

Deployment Steps

1. Deploy Single Agent Components

```
cd agents/budget_agent
sam deploy --stack-name budget-agent --region $AWS_R
```

2. Deploy Multi-Agent System

```
cd agents/financial_analysis
sam deploy --stack-name financial-analysis-agent --r

cd ../orchestrator
sam deploy --stack-name orchestrator-agent --region
```

3. Deploy Gateway and Observability

```
cd gateway
sam deploy --stack-name agentcore-gateway --region $

cd ../monitoring
sam deploy --stack-name agent-monitoring --region $A
```

4. Run Validation Tests

```
pytest tests/integration/ --region $AWS_REGION
```

Post-Deployment Validation

- Verify agent endpoints are accessible
- Confirm all agents respond correctly to test queries
- Validate multi-agent coordination
- Check CloudWatch metrics for expected activity
- Verify security logs are being generated

- Run load testing with 100 concurrent users

Troubleshooting Guide

Common Issues and Solutions

Issue	Symptoms	Resolution
Agent Not Responding	500 errors, timeouts	Check CloudWatch logs for execution errors
Tool Execution Failure	Agent responses indicate tool errors	Verify tool configuration and dependencies
Memory Issues	High memory utilization, container restarts	Increase memory allocation in Docker configuration
Security Policy Violations	Access denied errors	Review IAM policies and resource permissions
Poor Response Quality	Inconsistent or irrelevant responses	Review prompt engineering and guardrail configuration

Critical Path Troubleshooting

1. Agent Deployment Issues

```
aws logs get-log-events \  
--log-group-name "/aws/lambda/budget-agent" \  
--log-stream-name $(aws logs describe-log-streams  
--region $AWS_REGION
```

2. Bedrock Model Issues

```
aws bedrock get-model \  
--model-id "anthropic.claude-3-sonnet-20240229-v1:  
--region $AWS_REGION
```

3. Network Connectivity Issues

```
aws ec2 describe-vpcs --region $AWS_REGION  
aws ec2 describe-subnets --region $AWS_REGION
```

Next Steps

Immediate Actions

- Review this document with team members.
- Assign tasks based on expertise.
- Schedule daily standups for the first week of deployment.

- Provision AWS account and configure networking.
- Set up CI/CD pipeline and monitoring.
- Begin with single agent deployment and validate functionality.

Communication Plan

Activity	Frequency	Participants
Daily Standup	15 minutes daily	Entire team
Technical Review	Every Friday	Tech leads, architects
Client Update	Weekly	Project manager, client representatives
Risk Review	Bi-weekly	Tech leads, project manager

Contact Information

Primary Technical Lead

Name: Kashif Jamal

Email: K_Jamal@cloudage.co.in

Phone: +918087201590

Backup Technical Lead

Name: Sufyan Shaikh

Email: S_Shaikh@cloudage.co.in

Phone: +919028705740

Project Manager

Name: Rahul More

Email: R_More@cloudage.co.in

Phone: +971585786883

24/7 On-Call Rotation

- Week 1: Rahul More
- Week 2: Sufyan Shaikh
- Week 3: Kashif Jamal

Appendix A: Dependencies

Required AWS Services

Service	Purpose	Configuration
Amazon Bedrock	Foundation models for agents	Standard pricing tier
AWS Lambda	Serverless compute for agent logic	2048MB memory, 15 min timeout
Amazon DynamoDB	Conversation state storage	On-demand capacity
Amazon S3	Document storage	Standard storage class
Amazon API Gateway	Agent endpoints	REST API, standard tier
Amazon CloudWatch	Monitoring and logging	Standard tier
AWS Secrets Manager	Secure credential storage	Standard tier
AWS KMS	Data encryption	Default key

Required Software Versions

Component	Version
Python	3.12.3

Component	Version
AWS CLI	2.19.0
AWS SAM CLI	1.100.0
Docker	24.0.7
Node.js	18.17.1

Appendix B: Deployment Timeline

Phase	Duration	Key Milestones
Phase 1: Foundation Setup	2 days	AWS account configured, CI/CD pipeline established
Phase 2: Single Agent Deployment	3 days	Budget Agent deployed, custom tools implemented, basic monitoring in place
Phase 3: Multi-Agent System	4 days	Financial Analysis Agent deployed, Orchestrator Agent configured, communication verified

Phase	Duration	Key Milestones
Phase 4: Integration & Security	3 days	Gateway integration complete, security controls validated, compliance documentation created
Phase 5: Testing & Validation	2 days	Load testing completed, security scan passed, final validation report

About CloudAge

CloudAge Global Services is an AWS Advanced Consulting Partner with deep experience in AI/ML, cloud-native architecture, and financial services implementations. The team brings strong delivery capabilities, security expertise, and a track record of enterprise-scale AI deployments.

CloudAge Capabilities

- AI/ML solution development across highly regulated industries.
- Cloud architecture, migration, and security engineering.
- DevOps, site reliability engineering, and managed production operations.
- Data, analytics, and governance for modern financial systems.
- Security, compliance, and enterprise-grade operational readiness.

Why Choose CloudAge

- AWS Advanced Consulting Partner status.
- 100+ successful AI/ML deployments.
- Deep financial services domain expertise.
- 24/7 global delivery and on-call support.
- Commitment to innovation, quality, and secure operations.

CloudAge Solutions Architecture Team • Last Updated: June 03, 2026 •
Version 1.2

This document is for internal use only. Unauthorized distribution is prohibited.